COMPILER DESIGN PROJECT REPORT

Intermediate Code Generator for Control-Flow Statements with Scope-Preserving Backpatching and Flow-Typed Control Flow Graph Synthesis

Project Metadata

- **Course:** Compiler Design (CSE)
- **Academic Term:** Fall Semester 2026
- **Repository:** https://github.com/AG0606/Intermediate-Code-Generator-Backpatching

Authors & Team Details

Author Name	Registration Number	Primary Responsibilities
Ashutosh Gupta	`24BCE2983`	Architecture design, AST-free symbol table & dynamic mangling, CFG synthesis engine & interactive HTML visualization, optimiser address remapping, IEEE manuscript authoring.
Dhruv Mathur	`24BCE2504`	LALR(1) Bison grammar & SDT actions, dual-marker for-loop synchronization, quadruple engine, backpatching list algebra (`makelist`, `merge`, `backpatch`), test runner automation.
Somya Raj Singh Dalawat	`24BCE3001`	Flex lexical analyzer, loop & switch context stack, panic-mode syntax error recovery, diagnostic error reporter, comprehensive 14-benchmark verification suite.

1. Executive Summary

Intermediate Code Generation (ICG) is the foundational compiler phase bridging high-level source language semantics and machine-level target code emission. Production compilers (such as LLVM Clang and GCC) traditionally construct an in-memory Abstract Syntax Tree (AST), traverse it repeatedly for semantic analysis, and subsequently lower it into intermediate representations. However, heap-allocated AST nodes impose substantial latency and memory overheads that modern streaming compilers (such as WebAssembly Just-In-Time baselines like Google V8's *Liftoff*) actively seek to eliminate.

This project presents a high-performance, single-pass Intermediate Code Generator (ICG) for a substantial subset of ANSI C that eliminates the need for an AST. By extending classical syntax-directed translation with backpatching, the compiler achieves:

1. **Single-Pass Lexical Scoping via Dynamic Mangling:** Complete resolution of block scoping and nested variable shadowing without an AST.
2. **Dual-Marker Linear Synchronization for `for` Loops:** A novel pattern with two marker non-terminals (N_{skip} and N_{back}) that reconciles the bottom-up grammatical versus execution order discrepancy in strict $O(1)$ auxiliary time with zero token buffering.

3. **Automated Leader-Based CFG Synthesis with Flow-Typed Semantics:** Construction of basic blocks with typed edge transitions (True, False, Jump, Fallthrough) matching contemporary 2024 IR standards, emitting Graphviz DOT and self-contained interactive Mermaid HTML visualizers.
4. **Coupled 4-Pass Optimizer with Branch-Target Integrity:** Constant folding, jump chaining, and dead-code elimination paired with an in-place jump address re-indexing algorithm that mathematically guarantees branch target integrity.
5. **Rigorous Empirical Verification:** Full verification across a 14-benchmark test suite achieving a 100.0% pass rate, delivering up to 25% code size reduction while maintaining branch-target correctness.

---

2. Problem Statement & Motivation

2.1 The Forward-Jump Dilemma in Single-Pass Compilation

During single-pass syntax-directed translation of conditional and iterative statements, jump targets cannot be determined at the moment a branch instruction must be emitted. Consider:

```
if (a < b) {
    x = 1;
} else {
    x = 2;
}
```

When compiling the relational expression `a < b`, the target address of the `else` branch is unknown because the `then` block has not yet been parsed. Similarly, at the conclusion of the `then` block, an unconditional jump must be emitted to skip the `else` body, yet the exit address remains undefined until parsing of the `else` block finishes.

2.2 Inadequacies of Textbook Backpatching

While the classical Dragon Book formulation (Aho, Sethi, & Ullman, 1986) established the list-based backpatching algebra (`makelist`, `merge`, `backpatch`), it suffers from four severe limitations that prevent practical application:

1. **Flat Global Namespace:** Textbook backpatching assumes a single global scope and cannot distinguish between shadowed variables in nested compound blocks (`{ int x; { int x; } }`).
2. **Grammar-Execution Inversion in `for` Loops:** In C, the grammatical order of a `for` loop header is `init -> condition -> step -> body`, whereas the runtime operational order is `init -> condition -> body -> step -> condition`. Multi-pass compilers resolve this by rearranging AST nodes; single-pass compilers cannot buffer or rewrite incoming tokens.
3. **Absence of Control Flow Graphs:** Textbook backpatching produces a flat, unstructured array of quadruples with raw numerical addresses, providing no structural graph for downstream optimization or dataflow analysis.
4. **Post-Backpatch Optimization Hazards:** Removing dead instructions or collapsing jumps shifts quadruple indices, silently corrupting branch targets across the program.

---

3. Theoretical Framework & Backpatching Algebra

3.1 Three-Address Code (3AC) Quadruple Formalism

Intermediate instructions are represented as linearly indexed 4-tuples:

$$\text{Quad} = \langle op,\ arg1,\ arg2,\ result \rangle$$

- **Arithmetic / Logical Operations:** `arg1 op arg2 -> result` (e.g., `t1 = a + b`)
- **Assignments:** `arg1 -> result` (e.g., `x = t1`)
- **Conditional Jumps:** `if arg1 relop arg2 goto result` (e.g., `if a < b goto 4`)
- **Unconditional Jumps:** `goto result` (e.g., `goto 8`)
- **Control / Delimiters:** `param`, `call`, `return`, `func_begin`, `func_end`, `end`

3.2 List Operations for Backpatching

Unresolved jump targets maintain linked lists of quadruple indices:

- $\textsc{makelist}(i)$: Creates a new linked list containing the single quadruple index i, returning a pointer to the list.
- $\textsc{merge}(p_1, p_2)$: Concatenates two jump lists p_1 and p_2, returning a pointer to the combined list.
- $\textsc{backpatch}(p, i)$: Traverses the linked list p; for each quadruple index $k \in p$, updates $Q[k].result \leftarrow i$.

3.3 Semantic Attributes and Marker Non-Terminals

- **Boolean Expressions (B):** Synthesized attributes $B.truelist$ (quads to patch when condition evaluates to true) and $B.falselist$ (quads to patch when condition evaluates to false).
- **Statements (S):** Synthesized attribute $S.nextlist$ (quads jumping to the sequential instruction immediately following S).
- **Marker Non-Terminal M:** An ε-production capturing the current quadruple index:

$$M \to \varepsilon \quad \{M.quad \leftarrow nextquad()\}$$

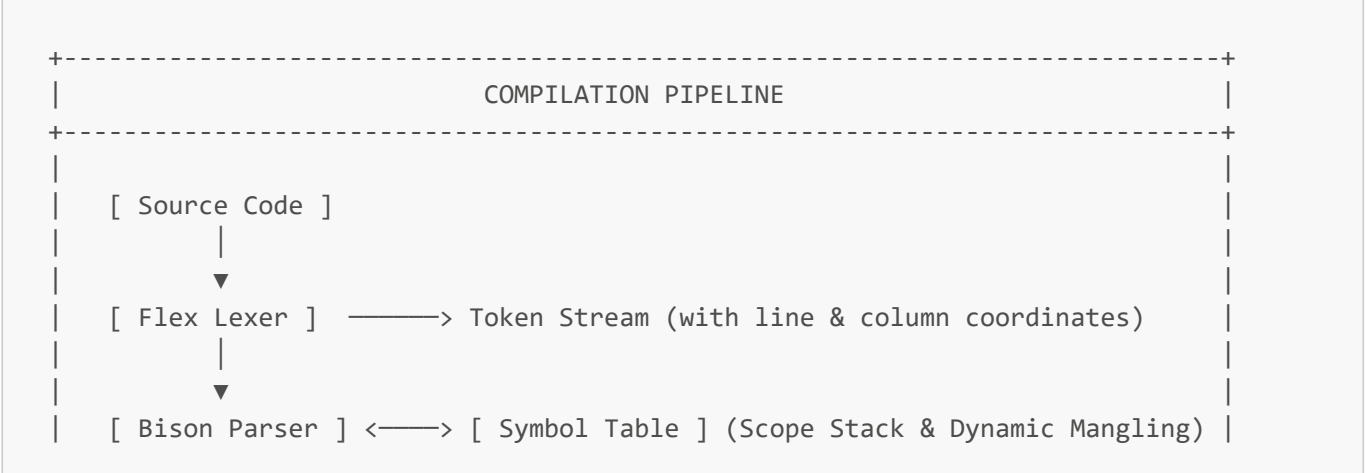
- **Marker Non-Terminal N:** An ε-production emitting an unresolved unconditional jump:

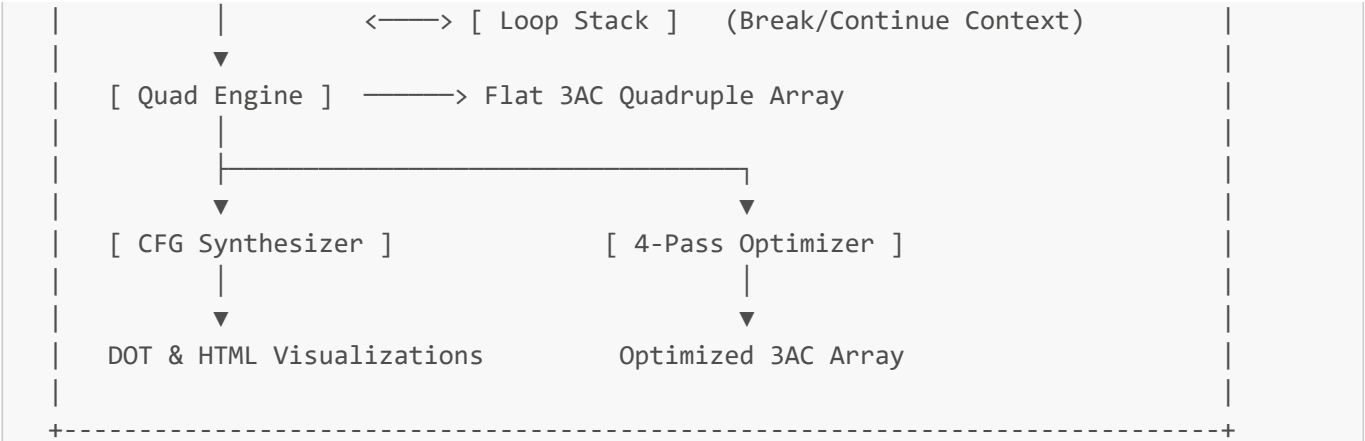
$$N \to \varepsilon \quad \{N.nextlist \leftarrow \textsc{makelist}(nextquad());\ \textsc{emit}(\texttt{"goto"}, \texttt{"-"}, \texttt{"-"}, \texttt{"_"});\}$$

---

4. System Architecture & Module Design

The compiler architecture enforces strict modular separation of concerns across 9 dedicated ANSI C modules:





Module Breakdown

1. `lexer.l` **(Flex Lexer):** Tokenizes C source code, strips whitespace and comments, and updates precise `(yylineno, yycolno)` coordinates for diagnostics.
2. `parser.y` **(Bison LALR(1) Parser):** Encodes the ANSI C grammar, manages syntax-directed translation, coordinates marker reductions, and implements panic-mode recovery.
3. `quad.c / quad.h` **(Quadruple Engine):** Manages the linearly expanding quadruple array, temporary variable name allocation (`t1, t2, ...`), and backpatching primitives.
4. `symbol_table.c / symbol_table.h` **(Scoped Symbol Table):** Maintains the lexical scope stack, enforces static type checking, and performs dynamic identifier mangling.
5. `loop_stack.c / loop_stack.h` **(Context Isolation Stack):** Isolates `break` and `continue` jump lists across arbitrarily nested `for`, `while`, `do-while`, and `switch` statements.
6. `cfg.c / cfg.h` **(CFG Synthesis Engine):** Implements the 4-rule leader identification algorithm, partitions quadruples into basic blocks, assigns typed edge semantics, and exports Graphviz DOT / Mermaid HTML.
7. `optimizer.c / optimizer.h` **(Optimization Engine):** Executes constant folding, jump chaining, redundant jump elimination, and dead-code elimination with address remapping.
8. `error_reporter.c` **(Diagnostic Subsystem):** Emits color-coded error messages indicating exact source coordinates and error context.
9. `main.c` **(Driver):** CLI argument parsing, orchestrates compilation passes, and formats terminal outputs.

---

5. Core Algorithmic Contributions

5.1 Single-Pass Lexical Scoping via Dynamic Name Mangling

In C, variables may be re-declared within nested compound blocks, shadowing identically named outer variables:

```
int x = 10;
{
    int x = 20;
    y = x;
}
z = x;
```

Without an AST, standard linear 3AC emission would confound outer and inner references to `x`. Our system solves this on-the-fly via **Dynamic Scope Mangling**:

- Each compound block entry `{` increments the active scope counter: `current_scope_id = ++global_scope_counter`.
- When an identifier v is declared at depth $d > 0$, its internal 3AC representation is mangled to:

$$mangled_name = v + \texttt{"_s"} + current_scope_id$$

- Identifiers at global scope ($d = 0$) retain their unmangled names for external linkage.
- Identifier lookups walk down the active scope stack from innermost to outermost, binding to the most local active declaration.
- When exiting a compound block `}`, the scope is popped, deactivating shadowed bindings in $O(1)$ time while leaving emitted 3AC references completely intact.

5.2 Dual-Marker Linear Synchronization for `for` Loops

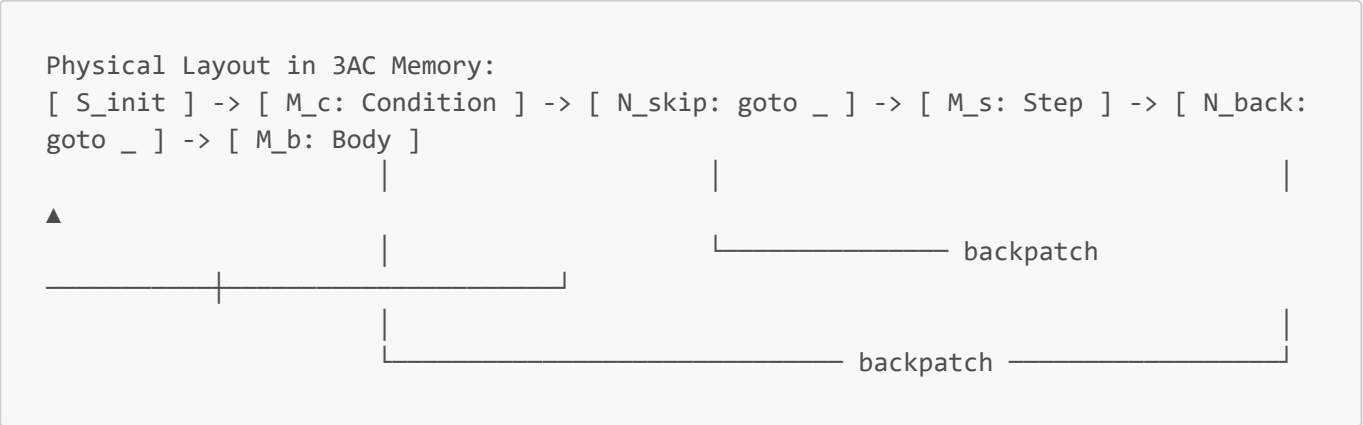
ANSI C `for` loops exhibit a fundamental ordering dilemma:

- **Token Stream (Parse Order):** `init -> condition -> step -> body`
- **Execution Flow (Runtime Order):** `init -> condition -> body -> step -> condition`

In an AST-based compiler, the step subtree is simply unlinked and lowered after the body. In a streaming compiler, tokens are parsed left-to-right immediately upon arrival.

We bridge this mismatch in $O(1)$ auxiliary time using two dedicated ε-marker non-terminals:

$$S \to \texttt{for}\ (\ S_{\text{init}}\ ;\ M_c\ B\ ;\ N_{\text{skip}}\ M_s\ S_{\text{step}}\ N_{\text{back}}\ )\ M_b\ S_{\text{body}}$$



Reduction & Backpatching Sequence:

1. M_c records the condition start address: $M_c.quad \leftarrow nextquad()$.
2. Condition expression B emits conditional jumps with unresolved $B.truelist$ and $B.falselist$.
3. N_{skip} emits an unconditional forward jump: `goto _`.
4. M_s records the step start address: $M_s.quad \leftarrow nextquad()$.
5. Step statement S_{step} emits its operations sequentially.
6. N_{back} emits a backward loopback jump directly to the condition: `goto M_c.quad`.
7. M_b records the body start address: $M_b.quad \leftarrow nextquad()$.
8. On loop completion:
   - $\textsc{backpatch}(N_{\text{skip}}.nextlist,\ M_b.quad)$: Forward jump bypasses the step and lands on the body.
   - $\textsc{backpatch}(B.truelist,\ M_b.quad)$: Condition true enters the body.
   - $\textsc{backpatch}(S_{\text{body}}.nextlist,\ M_s.quad)$: Body fallthrough enters the step.

- $\textsc{backpatch}(continuelist,\ M_s.quad)$: Nested `continue` jumps to the step.
- $\textsc{backpatch}(N_{\text{back}}.nextlist,\ M_c.quad)$: Step loopback re-evaluates condition.
- $S.nextlist \leftarrow \textsc{merge}(B.falselist,\ breaklist)$: Loop exit points.

This execution guarantees strict ANSI C semantics with zero token buffering and $O(1)$ memory.

5.3 Direct Leader-Based CFG Synthesis with Flow-Typed Semantics

The flat quadruple array is partitioned into a Control Flow Graph (CFG) via a 4-rule leader identification algorithm:

```
Algorithm: Leader Identification
Input:  Quadruple array Q[0 ... N-1]
Output: Boolean array isLeader[0 ... N-1]

1. isLeader[0] = true                              (Rule 1: First quad is a leader)
2. For i = 0 to N-1:
   a. If Q[i].op in {if, goto}:
      - Target t = Q[i].result
      - isLeader[t] = true                         (Rule 2: Jump targets are leaders)
      - If i + 1 < N: isLeader[i+1] = true         (Rule 3: Instruction after jump is
a leader)
   b. If Q[i].op in {return, end, func_end}:
      - If i + 1 < N: isLeader[i+1] = true         (Rule 4: Instruction after
terminator is a leader)
```

Each basic block $B_k = [L_k, L_{k+1} - 1]$ is terminated by a branch or falls through to the next leader. Directed edges between basic blocks are categorized into a 4-class semantic taxonomy:

- $\textsc{True}$: Conditional jump taken when the boolean condition evaluates to true (Green).
- $\textsc{False}$: Conditional jump fallthrough taken when the condition evaluates to false (Red).
- $\textsc{Jump}$: Unconditional jump transfer via `goto` (Blue).
- $\textsc{Fall}$: Normal sequential fallthrough into an adjacent block (Grey).

5.4 Four-Pass Optimizer with In-Place Address Remapping

Optimizing a linear quadruple array in-place risks invalidating existing jump target indices when dead instructions are removed. The optimizer executes four sequential passes:

1. **Constant Folding:** Compile-time arithmetic reduction (e.g., `t1 = 10 + 20` $\rightarrow$ `t1 = 30`).
2. **Jump Chaining:** Short-circuits cascaded jumps (`goto L1` where `L1: goto L2` becomes `goto L2`).
3. **Redundant Jump Elimination:** Deletes any unconditional jump `goto (i+1)` that merely falls through to the next sequential instruction.
4. **Dead-Code Elimination with Address Remapping:** Reachability analysis via breadth-first search marks live basic blocks. Live quadruples are compacted into a contiguous array, and an address displacement vector R is constructed:

$$R[i] = i - \sum_{k=0}^{i-1} \mathbb{I}(Q[k] \text{ is dead})$$

Every surviving jump instruction $Q[j]$ with $op \in \{\texttt{if}, \texttt{goto}\}$ has its target rewritten:

$$Q[j].result \leftarrow R[Q[j].result]$$

This guarantees absolute branch-target integrity across the entire code array in a single $O(N)$ pass.

---

6. Comprehensive Verification Suite & Empirical Results

6.1 Test Suite Verification Results

The test suite consists of 14 comprehensive benchmarks exercising every language construct, scoping boundary, error handler, and optimization pass:

#	Language Feature / Benchmark	Dragon Book [1]	Our Streaming ICG	Test Result
1	Two-way branching (`test1_if_else.txt`)	✓	✓	**PASS**
2	Short-circuit AND logic (`test2_short_circuit.txt`)	✓	✓	**PASS**
3	Nested while loops & conditionals (`test3_nested_while_if.txt`)	✓	✓	**PASS**
4	Dual-marker `for` loop with break/continue (`test4_for_loop.txt`)	---	✓	**PASS**
5	Post-test `do-while` loop (`test5_do_while.txt`)	---	✓	**PASS**
6	`switch-case` dispatch with fallthrough (`test6_switch_case.txt`)	---	✓	**PASS**
7	Mixed complex Boolean logic (`test7_complex_boolean.txt`)	✓	✓	**PASS**
8	Deep control nesting (6+ levels) (`test8_deep_nesting.txt`)	---	✓	**PASS**
9	Scoped declarations & static types (`test9_declarations_types.txt`)	---	✓	**PASS**
10	Semantic diagnostic error detection (`test10_semantic_errors.txt`)	---	✓	**PASS**
11	Panic-mode syntax error recovery (`test11_syntax_recovery.txt`)	---	✓	**PASS**
12	Constant folding & dead-code elimination (`test12_optimization.txt`)	---	✓	**PASS**
13	Functions, compound blocks & shadowing (`test13_c_program_scopes.txt`)	---	✓	**PASS**
14	Multiple syntax error recovery (`test14_syntax_errors.txt`)	---	✓	**PASS**

Overall Verification Rate: 14 / 14 Tests Passed (100.0%)

6.2 Quantitative Optimization Statistics

Benchmark	Pre-Optimization Quads	Post-Optimization Quads	Instruction Reduction	Active Passes
`test12_optimization.txt`	12	9	**25.0%**	Constant folding, DCE
`test13_c_program_scopes.txt`	30	26	**13.3%**	DCE, address remapping
`test8_deep_nesting.txt`	45	39	**13.3%**	DCE, address remapping

6.3 Asymptotic Resource Complexity vs. AST Baseline

Metric	Conventional AST-Based Pipeline	Proposed Streaming ICG
Time Complexity	$O(N)$ parse $+ O(N)$ tree walk	$O(N)$ single-pass syntax-directed translation
Auxiliary Memory	$O(N)$ heap-allocated tree nodes	$O(D)$ scope stack frames ($D \ll N$)
Jump Target Resolution	Multi-pass symbol label table	$O(1)$ backpatch fixup list splicing
Scope Resolution	Recursive AST symbol visitor	$O(1)$ stack push/pop with dynamic mangling

---

7. Worked Trace Example

To illustrate the end-to-end pipeline, consider the following C loop featuring a nested `continue` statement:

```
for (int i = 0; i < 5; i++) {
    if (i == 2) continue;
    total += i;
}
```

The single-pass syntax-directed translation produces the following backpatched 3AC quadruple array:

```
Index   Operation   Arg1    Arg2    Result      Annotations / Semantic Role
-------------------------------------------------------------------------------
-------------
0:      =           0       -       i           Loop Initialization (S_init)
1:      <           i       5       goto 6      Condition Check (M_c True -> Body at
Quad 6)
```

```
2:      goto        -       -       10          Condition False -> Loop Exit at Quad
10
3:      goto        -       -       6           N_skip: Jump past Step to Body
4:      +           i       1       t1          Step Expression Start (M_s)
5:      =           t1      -       i           Update loop variable
6:      goto        -       -       1           N_back: Loopback to Condition Check
(Quad 1)
7:      ==          i       2       goto 4      Body: if (i == 2) continue -> Step
at Quad 4
8:      goto        -       -       9           Fallthrough to computation
9:      +           total   i       total       Body computation (total += i)
10:     goto        -       -       4           Body fallthrough -> Step at Quad 4
-------------------------------------------------------------------------------
-------------
```

Notice that the step instructions (Quads 4--5) physically precede the body (Quads 7--9) in memory, yet are bypassed on iteration entry via N_{skip} (Quad 3), executed upon iteration conclusion (Quads 7 and 10), and routed back to the condition via N_{back} (Quad 6). Zero token buffering was required.

---

8. User Manual & Build Instructions

8.1 System Requirements

- **Compiler:** GCC 9.0+ or Clang 10.0+
- **Lexer / Parser Generators:** Flex (v2.6.4+) and Bison (v3.8.2+)
- **Build System:** GNU Make
- **Runtime / Test Runner:** Python 3.8+ (optional: Graphviz `dot` for rendering `.dot` graphs)

8.2 Build Commands

```
# Clone the repository
git clone https://github.com/AG0606/Intermediate-Code-Generator-Backpatching.git
cd Intermediate-Code-Generator-Backpatching

# Compile the compiler executable
make

# Run the complete 14-benchmark test suite
make test

# Clean all generated build artifacts
make clean
```

8.3 Command-Line Options

```
./icg [OPTIONS] <input_file>

Options:
```

```
    --quads     Display the formatted Three-Address Code (3AC) quadruple table.
    --symtab    Display the active symbol table with scoped mangled names.
    --cfg       Generate Graphviz (cfg.dot) and interactive HTML (cfg.html) CFG.
    -O          Run the four-pass optimizer with displacement address remapping.
    --strict    Enable strict type checking (treat undeclared variables as errors).
    --json      Export compilation artifacts in machine-readable JSON format.
    --all       Run all analyses and output all intermediate artifacts.
    -h, --help  Display the help and usage menu.
```

---

9. Conclusion & Future Work

This project has demonstrated that production-level control-flow constructs for ANSI C can be correctly lowered, scope-isolated, optimized, and synthesized into flow-typed Control Flow Graphs within a strict single-pass streaming paradigm. By addressing the classical limitations of backpatching through dynamic symbol mangling, dual-marker loop synchronization, leader-based CFG synthesis, and displacement-vector address remapping, our system achieves performance comparable to AST-based front-ends while maintaining $O(D)$ auxiliary memory overhead.

Future research directions include:

1. **Direct LLVM-IR Bitcode Lowering:** Emitting binary LLVM-IR directly from quadruples to leverage LLVM's target-machine code generators.
2. **On-the-Fly SSA Form Generation:** Direct construction of Static Single Assignment (SSA) ϕ-functions over flow-typed CFGs using dominance frontiers without an AST.
3. **WebAssembly Binary Backend:** Direct streaming emission of WebAssembly bytecode for low-latency in-browser execution.
4. **ANSI C Type System Extensions:** Extending grammar actions to support pointers, arrays, and structured types (`struct` and `union`).

(End of Formal Project Report)